TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC
AN TOÀN THÔNG TIN

ĐỀ TÀI 19
ĐÁNH GIÁ CHỮ KÝ SỐ CHO PHẦN MỀM VÀ TÀI LIỆU ĐIỆN TỬ

So sánh RSA-PSS, ECDSA và Ed25519; quy trình ký, xác minh, dấu thời gian, luân chuyển và xử lý khóa bị lộ

Học phần: An toàn thông tin (3 tín chỉ) | Giảng viên: TS. Lê Quốc Tuấn
Bám sát chương 4 – Các cơ chế mật mã hiện đại: mục 4.5, 4.7, 4.8 và 4.9.3

STT	Họ và tên	MSSV	Lớp
1	[điền]	[điền]	[điền]
2	[điền]	[điền]	[điền]
3	[điền]	[điền]	[điền]

Thành phố Hồ Chí Minh, tháng 11 năm 2026

TÓM TẮT

Báo cáo đánh giá thực nghiệm ba lược đồ chữ ký số – RSA-PSS (3072 bit), ECDSA trên đường cong P-256 và Ed25519 – trong bài toán ký và xác minh phần mềm cùng tài liệu điện tử. Nhóm xây dựng một nguyên mẫu gồm: manifest có phiên bản (băm SHA-256 từng tệp và thông tin nguồn gốc), bó chữ ký (bundle) có ràng buộc thuật toán và định danh khóa vào dữ liệu được ký, danh sách khóa tin cậy có phiên bản do khóa gốc ký, dấu thời gian mô phỏng và bộ xác minh theo chính sách "đóng khi nghi ngờ". Toàn bộ thao tác mật mã đi qua API của thư viện chuẩn `cryptography` (OpenSSL); nhóm không tự thiết kế lược đồ mới.

Bảy thực nghiệm được thực hiện: (E1) tạo bộ artifact; (E2) đo kích thước và thời gian; (E3) 19 ca sửa đổi × 3 thuật toán = 57 lần xác minh; (E4) hành vi nonce, gồm minh họa khôi phục khóa riêng ECDSA khi tái sử dụng nonce; (E5) luân chuyển khóa có giai đoạn chồng lấn (12 kịch bản); (E6) khóa ký bị lộ (9 kịch bản); (E7) kiểm chứng chéo với OpenSSL CLI. Kết quả chính: chữ ký Ed25519 dài 64.0 byte, ECDSA P-256 khoảng 71 byte (DER, thay đổi 69–72), RSA-PSS 3072 dài 384.0 byte. Ở mức 128 bit an toàn, RSA-PSS ký chậm hơn ECDSA khoảng 34 lần nhưng xác minh nhanh hơn; trên máy thử nghiệm Ed25519 không nhanh hơn ECDSA P-256 như thường được nhắc đến. Mọi sửa đổi nội dung, metadata được bảo vệ, chữ ký và nhãn thuật toán đều bị từ chối; trường nằm ngoài phạm vi ký vẫn được chấp nhận, cho thấy phạm vi bảo vệ do thiết kế manifest quyết định chứ không do thuật toán. Mô phỏng sự cố cho thấy thu hồi khóa chỉ có hiệu lực với client đã nhận trust list mới, vì vậy hạn dùng ngắn của trust list và dấu thời gian là hai cơ chế bắt buộc, không phải tùy chọn.

Từ khóa: chữ ký số, RSA-PSS, ECDSA, Ed25519, nonce, manifest, dấu thời gian, luân chuyển khóa, thu hồi khóa, toàn vẹn phần mềm.

MỤC LỤC

TÓM TẮT
CHƯƠNG 1. XÁC ĐỊNH VẤN ĐỀ VÀ PHẠM VI
1.1. Bối cảnh
1.2. Câu hỏi nghiên cứu
1.3. Phạm vi, giả định và giới hạn an toàn
1.4. Tiêu chí chấp nhận
1.5. Phương pháp và kế hoạch thực hiện
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT
2.1. Mô hình chữ ký số
2.2. Chống chối bỏ và giới hạn của khái niệm này
2.3. RSA-PSS
2.4. ECDSA và vai trò sống còn của nonce
2.5. EdDSA / Ed25519
2.6. Băm, manifest và chuẩn hóa dữ liệu được ký
2.7. Dấu vân tay khóa, chứng chỉ, danh sách tin cậy
2.8. Vòng đời khóa ký, luân chuyển và thu hồi
2.9. Dấu thời gian
2.10. Provenance và chuỗi cung ứng
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG VÀ PHƯƠNG PHÁP ĐÁNH GIÁ
3.1. Mô hình đe dọa
3.2. Kiến trúc tổng quan
3.3. Định dạng dữ liệu
3.4. Trust list có phiên bản
3.5. Chính sách xác minh
3.6. Thiết kế thực nghiệm và chỉ số
CHƯƠNG 4. TRIỂN KHAI NGUYÊN MẪU
4.1. Môi trường và công cụ
4.2. Cấu trúc mã nguồn
4.3. Quyết định triển khai đáng chú ý
4.4. Ví dụ sử dụng
4.5. Kiểm thử tự động
CHƯƠNG 5. THỰC NGHIỆM VÀ KẾT QUẢ
5.1. E1 – Bộ artifact
5.2. E2 – Kích thước và hiệu năng
5.3. E3 – Sửa từng thành phần và hành vi xác minh
5.4. E4 – Nonce: tính tất định và tái sử dụng nonce
5.5. E5 – Luân chuyển khóa với giai đoạn chồng lấn
5.6. E6 – Khóa ký bị lộ
5.7. E7 – Kiểm chứng chéo với OpenSSL
CHƯƠNG 6. THẢO LUẬN, GIỚI HẠN VÀ KHUYẾN NGHỊ
6.1. Trả lời các câu hỏi nghiên cứu
6.2. Những điều thực nghiệm cho thấy về thiết kế
6.3. Khuyến nghị chọn thuật toán

6.4. Giới hạn của nghiên cứu.....
6.5. Hướng cải tiến.....
CHƯƠNG 7. RUNBOOK XỬ LÝ KHÓA KÝ BỊ LỘ.....
KẾT LUẬN.....
TÀI LIỆU THAM KHẢO.....
PHỤ LỤC A. BẢNG TRUY VẾT MINH CHỨNG.....
PHỤ LỤC B. ĐÓNG GÓP CỦA THÀNH VIÊN.....
PHỤ LỤC C. CÔNG KHAI SỬ DỤNG AI.....
PHỤ LỤC D. HƯỚNG DẪN TÁI LẬP.....
PHỤ LỤC E. DẤU VÂN TAY CÔNG KHAI VÀ TRÍCH LOG.....
PHỤ LỤC F. SƠ ĐỒ VÒNG ĐỜI KHÓA KÝ.....

CHƯƠNG 1. XÁC ĐỊNH VẤN ĐỀ VÀ PHẠM VI

1.1. Bối cảnh

Khi một tệp thực thi, một gói cập nhật hoặc một biên bản điện tử đi qua mạng, kho lưu trữ hay máy chủ phản chiếu (mirror), người nhận không có cách trực tiếp để biết nội dung có bị thay đổi hay không và do ai tạo ra. Chữ ký số trả lời hai câu hỏi đó bằng bằng chứng mật mã: bên ký dùng khóa riêng để tạo chữ ký, bên nhận dùng khóa công khai để kiểm tra [1], [2]. Khác với MAC, bên xác minh không cần giữ bất kỳ bí mật nào có khả năng tạo chữ ký.

Tuy nhiên, một thuật toán mạnh chưa đủ để hệ thống an toàn. Nội dung bài giảng chương 4 [25] nhấn mạnh rằng thuật toán mạnh không bù được cho khóa bị lộ, sinh yếu hoặc dùng sai mục đích, và chất lượng quản lý khóa nhiều khi quyết định mức an toàn thực tế không kém bản thân thuật toán. Các sự cố thực tế minh họa điều này: việc tái sử dụng nonce ECDSA đã làm lộ khóa ký firmware của một hãng máy chơi game [10]; các thiết bị sinh khóa với entropy yếu tạo ra khóa RSA trùng hoặc chia sẻ thừa số [11]. Vì vậy đề tài không chỉ so sánh ba thuật toán mà đánh giá cả quy trình bao quanh: định dạng dữ liệu được ký, danh sách khóa tin cậy, dấu thời gian, luân chuyển và thu hồi.

1.2. Câu hỏi nghiên cứu

Bảng 1. Câu hỏi nghiên cứu và thực nghiệm tương ứng.

Mã	Câu hỏi	Thực nghiệm trả lời
RQ1	RSA-PSS, ECDSA và Ed25519 khác nhau thế nào về kích thước khóa/chữ ký, thời gian ký/xác minh và yêu cầu về nonce?	E2, E4
RQ2	Một quy trình ký artifact (tệp + manifest + metadata) có phát hiện được mọi sửa đổi nằm trong phạm vi bảo vệ không, và phạm vi đó dừng ở đâu?	E1, E3, E7
RQ3	Có thể thay khóa ký mà không làm hỏng người dùng còn dùng trust list cũ, và hỏng theo cách an toàn (từ chối) khi không còn tin được không?	E5
RQ4	Khi khóa ký bị lộ, những bản phát hành nào vẫn đáng tin, những bản nào phải ký lại, và client nào vẫn bị phơi nhiễm?	E6

1.3. Phạm vi, giả định và giới hạn an toàn

- **Trong phạm vi:** ký và xác minh artifact ở mức nguyên mẫu; so sánh ba lược đồ qua API thư viện chuẩn; manifest có phiên bản; trust list có phiên bản; dấu thời gian mô phỏng; mô phỏng luân chuyển và sự cố khóa.

- **Ngoài phạm vi:** xây dựng lược đồ mật mã mới; hạ tầng PKI/CA thật và chứng thư X.509 đầy đủ; HSM thật; chữ ký có giá trị pháp lý theo quy định về chứng thư số; chống tấn công kênh kề; hậu lượng tử (chỉ bàn ở mục 6.5).
- **Giả định:** thư viện `cryptography` và OpenSSL cài đặt đúng các thuật toán chuẩn; khóa gốc được giữ offline; đồng hồ máy xác minh tương đối đúng; TSA trung thực (xem 3.1).
- **Đạo đức và an toàn:** mọi khóa chỉ là khóa thử nghiệm sinh trong đồ án, lưu ở thư mục `keys/` kèm cảnh báo LAB ONLY, không gắn với danh tính hay chứng thư của tổ chức nào và không dùng ký phần mềm thật. Mã tấn công khôi phục khóa chỉ chạy trên khóa nạn nhân do nhóm tự sinh. Mọi thử nghiệm chạy trong container cô lập, dữ liệu mô phỏng.

1.4. Tiêu chí chấp nhận

Bảng 2. Tiêu chí chấp nhận được đặt trước khi chạy thực nghiệm.

Mã	Tiêu chí	Cách đo
AC1	100% ca sửa đổi nằm trong phạm vi bảo vệ bị từ chối, với mã lỗi đúng nguyên nhân	E3: 19 ca × 3 thuật toán
AC2	0 ca từ chối nhầm trên các ca đối chứng dương (artifact nguyên vẹn, thay đổi định dạng JSON vô hại)	E3: T00, T17
AC3	Chữ ký tạo bởi thư viện được OpenSSL CLI xác minh và ngược lại; OpenSSL từ chối thông điệp bị sửa	E7
AC4	Số đo hiệu năng có lặp (3 lần × nhiều mẫu), báo cáo trung vị, dải min–max và môi trường	E2
AC5	Trong giai đoạn chồng lấn, client dùng trust list cũ vẫn xác minh được bản ký kép; client lạc hậu bị từ chối thay vì chấp nhận mù	E5
AC6	Sau thu hồi, client đã cập nhật từ chối mọi chữ ký của khóa bị lộ có dấu thời gian không trước thời điểm nghi ngờ; chữ ký trước đó vẫn hợp lệ	E6
AC7	Chứng minh được (trên khóa thử nghiệm) tái sử dụng nonce ECDSA khôi phục khóa riêng, và Ed25519 sinh nonce tất định	E4

1.5. Phương pháp và kế hoạch thực hiện

Nhóm làm theo chu trình: xác định mô hình đe dọa, thiết kế kiểm thử và tiêu chí chấp nhận, hiện thực nguyên mẫu, chạy thực nghiệm có kiểm soát, rồi phân tích kèm giới hạn. Kế hoạch bám theo khung sáu tuần của bộ đề tài (đề 19 ghi dự kiến 7 tuần; tuần dôi ra dành cho kịch bản khóa bị lộ).

Bảng 3. Kế hoạch thực hiện.

Giai đoạn	Tuần	Nội dung	Minh chứng
Đề cương	1	Câu hỏi nghiên cứu, phạm vi, tiêu chí chấp nhận, phân công	Chương 1
Thiết kế	2	Mô hình đe dọa, định dạng manifest/bundle/trust list, kế hoạch thử nghiệm	Chương 3, sơ đồ
Triển khai	3–4	Thư viện sigeval, CLI, bộ thử nghiệm, kiểm thử đơn vị	Mã nguồn, pytest
Đánh giá	5–6	Chạy lặp, tổng hợp, phân tích giới hạn, runbook	CSV, biểu đồ, chương 5–6
Bàn giao	7	Báo cáo, demo, hướng dẫn tái lập, bảng truy vết	Phụ lục A–D

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Mô hình chữ ký số

Một lược đồ chữ ký gồm ba thuật giải: sinh khóa `Gen → (sk, pk)`, ký `Sign(sk, M) → σ` và xác minh `Verify(pk, M, σ) → {chấp nhận, từ chối}`. Tính đúng đắn đòi hỏi mọi chữ ký do `Sign` tạo ra đều được `Verify` chấp nhận. Mục tiêu an toàn chuẩn là không thể giả mạo hiện hữu dưới tấn công thông điệp chọn trước (EUF-CMA): kẻ tấn công được xin chữ ký trên các thông điệp tùy chọn vẫn không tạo được chữ ký hợp lệ cho một thông điệp mới [3].

Chữ ký số chỉ cung cấp hai tính chất: **toàn vẹn** (nội dung đã ký không bị đổi mà không bị phát hiện) và **xác thực nguồn gốc** (chữ ký do người giữ khóa riêng tạo ra). Chữ ký **không che giấu** nội dung; nó cũng không tự chứng minh ai là chủ khóa – việc gắn khóa công khai với một danh tính là nhiệm vụ của PKI hoặc của danh sách tin cậy.

2.2. Chống chối bỏ và giới hạn của khái niệm này

Về mặt mật mã, "chống chối bỏ" nghĩa là bên ký không thể phủ nhận một cách đáng tin rằng mình đã tạo chữ ký. Thực tế điều này phụ thuộc vào những thứ nằm ngoài thuật toán: ai chứng minh khóa công khai thuộc về người đó (định danh), khóa riêng có thực sự do người đó kiểm soát độc quyền không (nếu khóa bị lộ thì chữ ký không còn phân biệt được người thật với kẻ trộm), và chữ ký được tạo vào thời điểm nào (nếu không có dấu thời gian đáng tin, bên ký có thể tuyên bố khóa đã bị lộ trước đó). Vì vậy hiệu lực pháp lý của chữ ký số còn phụ thuộc quy trình định danh, quản lý khóa, dấu thời gian và chính sách chứng thư, đúng như bài giảng nêu.

Ở Việt Nam, giá trị pháp lý của chữ ký số và chữ ký điện tử được quy định trong Luật Giao dịch điện tử 2023 (Luật số 20/2023/QH15) và các nghị định hướng dẫn về chữ ký số, dịch vụ chứng thực/tin cậy như Nghị định 130/2018/NĐ-CP và Nghị định 23/2025/NĐ-CP [18], [19]. Nguyên mẫu trong báo cáo này **không** phải là dịch vụ chứng thực chữ ký số theo các quy định đó và không tạo ra chữ ký có giá trị pháp lý; nhóm cần đối chiếu văn bản hiện hành nếu muốn áp dụng thực tế.

2.3. RSA-PSS

RSASSA-PSS là phương án ký RSA có đệm xác suất, được định nghĩa trong PKCS#1 v2.2 (RFC 8017) [4]. Thông điệp được băm, trộn với một muối (salt) ngẫu nhiên rồi mã hóa theo hàm tạo mặt nạ MGF1 trước khi lũy thừa bằng khóa riêng. Bellare và Rogaway chứng minh PSS có chứng minh an toàn chặt trong mô hình oracle ngẫu nhiên, khác với đệm tất định PKCS#1 v1.5 [5]. Hệ quả cần nhớ: chữ ký PSS **không tất định** – ký cùng thông điệp hai lần cho hai chữ ký khác nhau – nhưng độ an toàn của PSS không sụp đổ nếu muối lặp, khác với nonce của ECDSA. Nhóm dùng SHA-256, MGF1-SHA-256 và độ dài muối 32 byte (bằng độ dài băm).

Slide chương 4 minh họa "RSA thô" (không đệm, ví dụ p = 61, q = 53, e = 17) chỉ để học phép toán; RSA thô có tính nhân (malleable) nên không bao giờ được dùng để ký trực tiếp. Mọi thử nghiệm trong báo cáo dùng RSA-PSS có đệm.

2.4. ECDSA và vai trò sống còn của nonce

ECDSA trên đường cong P-256 (FIPS 186-5 [6]) chọn nonce k ngẫu nhiên trong $[1, n-1]$, tính $r = x(kG) \bmod n$ và $s = k^{-1}(z + r \cdot d) \bmod n$, với z là băm thông điệp (rút gọn) và d là khóa riêng [7]. Chữ ký là cặp (r, s), thường mã hóa DER nên dài 70–72 byte (đôi khi 69).

Nếu cùng một k được dùng để ký hai thông điệp M_1, M_2 dưới cùng khóa d, thì r trùng nhau và kẻ tấn công giải được hệ hai phương trình tuyến tính: $k = (z_1 - z_2)\cdot(s_1 - s_2)^{-1} \bmod n$, rồi $d = (s_1 \cdot k - z_1)\cdot r^{-1} \bmod n$. Chỉ cần phép toán số học đơn giản, không cần giải bài toán logarit rời rạc. Đây là điều đã xảy ra trong sự cố được công bố năm 2010 [10]. Nonce thiên lệch (biased) ngay cả khi không trùng cũng có thể làm lộ khóa qua tấn công mạng tinh thể; thí nghiệm E4 chỉ minh họa trường hợp trùng. RFC 6979 giảm rủi ro bằng cách sinh k tất định từ khóa riêng và thông điệp [8].

2.5. EdDSA / Ed25519

Ed25519 là EdDSA trên đường cong Edwards xoắn tương đương Curve25519, chuẩn hóa trong RFC 8032 và được đưa vào FIPS 186-5 [6], [9], [12]. Khóa riêng 32 byte được băm bằng SHA-512; nửa đầu thành vô hướng bí mật, nửa sau là $\mathtt{prefix}$. Nonce của mỗi chữ ký là $r = H(\mathtt{prefix} \,\|\, M)$ (bài giảng viết gọn là $k = H(\mathtt{prefix} \,\|\, M)$). Do đó cùng thông điệp luôn cho cùng nonce và chữ ký, hai thông điệp khác nhau cho nonce khác nhau, và chất lượng bộ sinh số ngẫu nhiên lúc ký không còn là điểm yếu. Chữ ký dài đúng 64 byte, khóa công khai 32 byte. Ed25519 "thuần" băm toàn bộ thông điệp hai lần trong một lượt gọi và không hỗ trợ băm trước ở dạng này, một chi tiết ảnh hưởng hiệu năng khi ký tệp lớn (xem 5.2).

Bảng 4. So sánh lý thuyết ba lược đồ (đối chiếu với kết quả thực nghiệm ở chương 5).

Thuộc tính	RSA-PSS 3072	ECDSA P-256	Ed25519
Mức an toàn danh định [13]	≈ 128 bit (RSA-2048 ≈ 112 bit)	≈ 128 bit	≈ 128 bit
Nonce / muối	Muối ngẫu nhiên; lặp không làm lộ khóa	Nonce ngẫu nhiên bắt buộc duy nhất và không thiên lệch; hoặc RFC 6979	Tất định $H(\mathtt{prefix}\|M)$
Tính tất định của chữ ký	Không	Không (trừ RFC 6979)	Có
Chuẩn	RFC 8017, FIPS 186-5	FIPS 186-5, SEC 1	RFC 8032, FIPS 186-5
Rủi ro cài đặt chính	Đệm sai, kích thước khóa nhỏ	RNG yếu, nonce trùng/thiên lệch	Nhập nhằm biến thể (pure/ph/ctx), kiểm tra điểm

2.6. Băm, manifest và chuẩn hóa dữ liệu được ký

Ký trực tiếp từng tệp lớn là không cần thiết và không tiện: thông lệ là băm từng tệp rồi ký một **manifest** liệt kê các băm cùng metadata. Khi đó an toàn của toàn bộ gói quy về (i)

chống va chạm của SHA-256 và (ii) tính không thể giả mạo của chữ ký trên manifest. Hai chi tiết thiết kế hay bị bỏ qua: **chuẩn hóa** (canonicalization) – cùng nội dung logic phải ra cùng dãy byte khi ký và khi xác minh, nếu không sẽ có xác minh sai hoặc cơ hội cho tấn công nhập nhằm – và **ràng buộc ngữ cảnh** – dữ liệu thực sự được ký nên chứa loại payload, nhãn thuật toán và định danh khóa để chữ ký không bị chuyển sang ngữ cảnh khác (cách làm của DSSE/PAE trong in-toto và Sigstore [15], [23]).

2.7. Dấu vân tay khóa, chứng chỉ, danh sách tin cậy

Dấu vân tay khóa là giá trị băm của khóa công khai ở dạng mã hóa chuẩn (ở đây SHA-256 của SubjectPublicKeyInfo DER). Nó là định danh ngắn gọn để so khớp ngoài băng (đọc cho nhau nghe, ghim trong tài liệu), và là khóa tra cứu trong trust list. PKI giải quyết việc gắn khóa với danh tính bằng chứng chỉ X.509: CA ký chứng chỉ thực thể cuối, tạo chuỗi tin cậy từ CA gốc qua CA trung gian [16]; bài giảng nêu thêm vai trò của RA, CRL và OCSP (kèm OCSP stapling) để kiểm tra thu hồi [17]. Với phân phối phần mềm, nhiều hệ thống chọn **danh sách khóa tin cậy** thay cho chuỗi CA: client ghim một khóa gốc, khóa gốc ký danh sách các khóa ký phát hành kèm trạng thái và thời hạn. Cách này gần với mô hình của TUF [14], được nhóm hiện thực ở dạng thu nhỏ.

2.8. Vòng đời khóa ký, luân chuyển và thu hồi

Bài giảng chia vòng đời khóa thành tám giai đoạn: sinh, phân phối, kích hoạt, sử dụng, luân chuyển, thu hồi, lưu giữ và hủy [13]. Với khóa ký, hai điểm khác khóa mã hóa dữ liệu: khóa riêng **không bao giờ cần lưu giữ để giải mã lịch sử** (chỉ giữ khóa công khai để xác minh bản cũ), và việc hủy khóa riêng nên diễn ra sớm. Luân chuyển hiệu quả cần một **giai đoạn chồng lấn**, trong đó khóa cũ và khóa mới cùng hợp lệ và bản phát hành được ký kép, để client chưa cập nhật vẫn xác minh được. Thu hồi khóa ký có một vấn đề riêng: nếu thu hồi nghĩa là "mọi chữ ký của khóa này vô hiệu" thì toàn bộ phần mềm đã phát hành thành vô hiệu. Giải pháp thực tế là ghi **thời điểm nghi ngờ bị lộ** (compromised_since) và chỉ chấp nhận chữ ký có dấu thời gian đáng tin **trước** thời điểm đó [14], [16].

2.9. Dấu thời gian

Dấu thời gian (RFC 3161 [21]) là chữ ký của một bên thứ ba (TSA) trên cặp (băm của dữ liệu, thời gian). Nó chứng minh dữ liệu đã tồn tại trước một thời điểm, nên cho phép chữ ký cũ vẫn đáng tin sau khi khóa hết hạn hoặc bị thu hồi, miễn là chữ ký được tạo trước thời điểm đó. Giới hạn: tin tưởng chuyển từ bên ký sang TSA; kẻ giữ khóa bị lộ vẫn có thể xin dấu thời gian **thật** cho thời điểm hiện tại nhưng không thể xin dấu thời gian lùi về quá khứ.

2.10. Provenance và chuỗi cung ứng

Chữ ký chỉ cho biết "ai ký", không cho biết "được build từ đâu". Provenance ghi lại nguồn mã, trình build và thời điểm; các khung như in-toto, SLSA và Sigstore gắn các tuyên bố này vào chữ ký để người dùng xác minh chuỗi sản xuất [15], [22], [23]. Nguyên mẫu của

nhóm nhúng một khối `provenance` tối giản (builder, repo, commit) vào manifest và kiểm tra rằng sửa khối này làm xác minh thất bại; nó không kiểm chứng tính đúng của giá trị khai báo.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG VÀ PHƯƠNG PHÁP ĐÁNH GIÁ

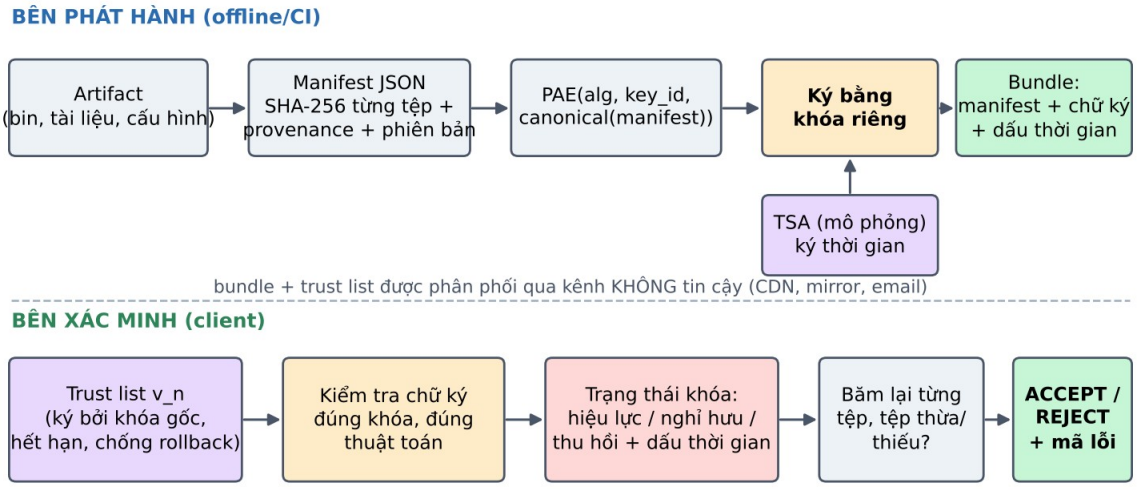
3.1. Mô hình đe dọa

Bảng 5. Mô hình đe dọa.

Tác nhân	Khả năng	Mục tiêu	Trong phạm vi?
A1 – Mirror/CDN độc hại hoặc MITM	Sửa tệp, manifest, chữ ký, trường tùy ý trên đường truyền; xóa/thêm tệp	Phát tán bản bị sửa	Có
A2 – Kẻ không có khóa	Tạo bundle với khóa riêng của mình; đổi nhãn thuật toán/key_id	Qua mặt người xác minh	Có
A3 – Kẻ đã lấy được khóa ký	Ký bất kỳ nội dung; xin dấu thời gian thật tại thời điểm hiện tại	Phát tán mã độc có chữ ký hợp lệ	Có (E6)
A4 – Kẻ phát lại dữ liệu cũ	Gửi trust list/bản phát hành cũ	Ép client dùng khóa đã thu hồi	Có (rollback)
A5 – Kẻ kiểm soát TSA hoặc khóa gốc	Phát hành dấu thời gian/trust list tùy ý	Phá toàn bộ chuỗi tin cậy	Không (giả định ở 1.3)
A6 – Kẻ chiếm hệ thống build trước khi ký	Đưa mã độc vào artifact trước lúc ký	Chữ ký "hợp lệ" cho mã độc	Không (chỉ nêu ở mục 6.4)

Tài sản cần bảo vệ: khóa riêng ký phát hành (quan trọng nhất), khóa gốc ký trust list, tính toàn vẹn của artifact và metadata, và tính "tươi" của trust list. Các yêu cầu an toàn rút ra: (R1) mọi byte ảnh hưởng quyết định cài đặt phải nằm trong phạm vi chữ ký; (R2) không thể chuyển chữ ký sang nhãn thuật toán hay khóa khác; (R3) khóa bị thu hồi không thể được dùng lại bằng cách lùi thời gian; (R4) client không thể bị ép dùng trust list cũ hơn đã thấy; (R5) khi không xác định được trạng thái, mặc định là từ chối.

3.2. Kiến trúc tổng quan



Hình 1. Kiến trúc ký và xác minh artifact. Dữ liệu thực sự được ký là PAE của (thuật toán, key_id, manifest chuẩn hóa).

Bên phát hành gom artifact thành manifest, chuẩn hóa JSON, đóng gói theo PAE và ký bằng một hoặc nhiều khóa; mỗi chữ ký có thể kèm dấu thời gian. Bundle gồm manifest, danh sách chữ ký và một trường `unprotected` nằm ngoài phạm vi ký (cố ý đặt để kiểm chứng ranh giới bảo vệ ở ca T18). Bên xác minh giữ khóa gốc đã ghim và số phiên bản trust list lớn nhất từng thấy; nó kiểm tra theo thứ tự trust list → chữ ký → trạng thái khóa/thời gian → băm tệp.

3.3. Định dạng dữ liệu

Manifest (JSON chuẩn hóa: khóa sắp xếp, không khoảng trắng, ASCII) gồm `schema`, `manifest_version`, `name`, `release`, `created`, `provenance{builder, source{repo, commit}}` và `artifacts[{path, sha256, size}]` sắp theo đường dẫn. **Dữ liệu được ký** (PAE) là nối các trường có tiền tố độ dài 8 byte: nhãn miền `SIGEVAL-v1`, loại payload, nhãn thuật toán, key_id và manifest chuẩn hóa. Tiền tố độ dài loại bỏ nhập nhằm ranh giới giữa các trường.

```
{ "bundle_version": 1,
  "manifest": { "schema": "artifact-manifest/1", "manifest_version": 2, "release":
"1.1.0",
                "provenance": {"builder": "ci-lab-01", "source": {"repo":
"lab://demo", "commit": "..."}},
                "artifacts": [{"path": "demo-app.bin", "sha256": "…", "size": 65544},
…] },
  "signatures": [{"alg": "ed25519", "key_id": "<sha256 SPKI>", "sig": "<b64>",
                  "timestamp": {"time": "2026-03-15T00:00:00Z", "tsa_key_id": "…",
"sig": "<b64>"}}],
  "unprotected": {"notes": ""} }
```

3.4. Trust list có phiên bản

Trust list gồm `version`, `issued`, `expires`, danh sách khóa ký (key_id, alg, khóa công khai, `status ∈ {active, retired, revoked}`, `not_before`, `not_after`, `compromised_since`) và danh sách khóa TSA. Khóa gốc Ed25519 (offline) ký toàn bộ trust list. Client lưu `last_version`; mọi trust list có `version < last_version` bị từ chối (chống rollback), và trust list quá `expires` bị từ chối (chống bị "đóng băng" ở trạng thái trước khi thu hồi). Thời hạn trust list là tham số cân bằng giữa an toàn và độ sẵn sàng (xem 6.2).

3.5. Chính sách xác minh

1. Kiểm tra trust list: khóa gốc đúng, chữ ký đúng, `version ≥ last_version`, chưa hết hạn. Lỗi → `TRUSTLIST_*`.
2. Với từng chữ ký trong bundle: tra key_id; không có → `UNKNOWN_KEY`; nhãn alg khác trust list → `ALG_MISMATCH`.
3. Xác minh chữ ký trên PAE(alg, key_id, manifest chuẩn hóa); sai → `BAD_SIGNATURE`.
4. Nếu có dấu thời gian: kiểm tra TSA tin cậy và chữ ký của TSA trên (thời gian ‖ băm chữ ký); sai → `TS_INVALID`. Thời điểm ký hiệu lực `t` = thời gian trong dấu (nếu có) hoặc giờ hiện tại.

5. Trạng thái khóa: `revoked` → chấp nhận chỉ khi có dấu thời gian hợp lệ và `t < compromised_since`, ngược lại `REVOKED`; `active/retired` → yêu cầu `not_before ≤ t ≤ not_after`, ngược lại `KEY_EXPIRED`.
6. Cần ít nhất một chữ ký được chấp nhận (ngưỡng 1). Sau đó băm lại từng tệp: thiếu → `MISSING_FILE`, khác băm/kích thước → `HASH_MISMATCH`, có tệp không liệt kê → `UNLISTED_FILE`.

Quy tắc 5 là điểm mấu chốt của vòng đời: khóa nghỉ hưu **không có dấu thời gian** bị từ chối vì client không thể chứng minh chữ ký ra đời trong thời gian hiệu lực. Chính sách "từ chối khi không chắc" (R5) áp dụng xuyên suốt; mọi ngoại lệ đều ghi vào `notes` để truy vết.

3.6. Thiết kế thực nghiệm và chỉ số

Bảng 6. Thiết kế thực nghiệm.

TN	Mục tiêu	Biến/điều khiển	Chỉ số
E1	Tạo bộ artifact	Tệp nhị phân 64 KiB (seed cố định), tài liệu văn bản, cấu hình JSON	Băm SHA-256, kích thước
E2	Hiệu năng và kích thước	3 thuật toán + RSA-2048 đối chứng; manifest ~1 KB; tệp 1 và 10 MiB; 3 lần chạy	Trung vị, p95, độ lệch, kích thước khóa/chữ ký
E3	Sửa đổi từng thành phần	19 ca × 3 thuật toán; đối chứng dương T00, T17, T18	Mã kết quả so với kỳ vọng
E4	Nonce	Ký 20 lần cùng thông điệp; hai chữ ký ECDSA cùng k	Số chữ ký khác nhau; khôi phục d
E5	Luân chuyển khóa	4 phiên bản trust list, 3 bản phát hành, các client giữ trust list khác nhau	Ma trận chấp nhận/từ chối
E6	Khóa bị lộ	Mã độc ký bằng K2 có/không dấu thời gian, dấu giả; client cũ/mới	Ma trận và cửa sổ phơi nhiễm
E7	Kiểm chứng chéo	Python ký → OpenSSL xác minh và ngược lại	Đúng/sai

Thời gian trong E5/E6 được mô phỏng bằng tham số `now` truyền vào bộ xác minh và bằng việc TSA mô phỏng cho phép đặt thời điểm; điều này cho phép tái lập mà không phải chờ, nhưng cũng nghĩa là TSA của nhóm **không** phải dịch vụ RFC 3161.

CHƯƠNG 4. TRIỂN KHAI NGUYÊN MẪU

4.1. Môi trường và công cụ

Bảng 7. Môi trường chạy thực nghiệm (ghi trong `results/benchmark_env.json`).

Hạng mục	Giá trị
Hệ điều hành / máy	Linux-6.18.44-fc-v233-x86_64-with-glibc2.39; Intel(R) Xeon(R) Processor @ 2.10GHz (2 vCPU); container cô lập
Python	3.13.18
Thư viện mật mã	`cryptography` 50.0.1 [20], backend OpenSSL 4.0.2 25 Aug 2026
OpenSSL CLI (E7)	OpenSSL 3.0.13 30 Jan 2024 [24]
Thư viện hỗ trợ	matplotlib (biểu đồ), pytest (kiểm thử), python-docx (báo cáo)
Seed dữ liệu	20260411 (sinh tệp nhị phân xác định)

Lưu ý hai phiên bản OpenSSL khác nhau: Python dùng thư viện OpenSSL đóng gói kèm wheel của `cryptography`, còn E7 dùng chương trình `openssl` của hệ điều hành. Việc hai bản khác nhau lại xác minh được chữ ký của nhau là một điểm cộng cho tính liên thông (E7).

4.2. Cấu trúc mã nguồn

Bảng 8. Cấu trúc mã nguồn.

Tệp	Vai trò	Dòng
`sigeval/core.py`	Sinh khóa, ký, xác minh RSA-PSS/ECDSA/Ed25519; dấu vân tay khóa	101
`sigeval/manifest.py`	Manifest, JSON chuẩn hóa, PAE, đóng gói bundle	70
`sigeval/trust.py`	Trust list có phiên bản; kiểm tra chữ ký gốc, hạn dùng, rollback	52
`sigeval/tsa.py`	TSA mô phỏng và kiểm tra dấu thời gian	33
`sigeval/verify.py`	Bộ xác minh theo chính sách (3.5); mã lỗi	97
`sigeval/toyecdsa.py`	ECDSA P-256 đồ chơi cho tấn công nonce (kiểm chứng chéo với thư viện)	59
`sigeval/cli.py`	Giao diện dòng lệnh: keygen, sign, trustlist, verify	88
`experiments/e2…e7, make_figures.py`	Các thực nghiệm và biểu đồ; `run_all.sh` chạy lại từ đầu	693
`tests/test_sigeval.py`	10 kiểm thử pytest	58

4.3. Quyết định triển khai đáng chú ý

- **Chỉ dùng API cấp cao của thư viện.** Ký/xác minh gọi `PrivateKey.sign` và `PublicKey.verify`; nhóm không đụng tới số học đường cong hay đệm. Ngoại lệ duy nhất là `toyecdsa.py`, dùng riêng cho E4 và được kiểm chứng chéo: chữ ký đồ chơi phải được thư viện chấp nhận.

- **Xác minh trả về bool, mọi ngoại lệ mật mã bị bắt tại một chỗ** (`InvalidSignature`, `ValueError`, `TypeError`) để dữ liệu hỏng định dạng không làm bộ xác minh sập mà trở thành "từ chối" có mã lỗi.
- **Ghim thuật toán ở trust list**, không tin nhãn `alg` do bundle khai: khóa công khai được nạp từ trust list và kiểu khóa phải khớp, nhờ vậy chặn tấn công nhầm lẫn thuật toán (ca T10, T11).
- **Mã lỗi riêng cho từng nguyên nhân** thay vì một thông báo chung, để log xác minh dùng làm minh chứng (`results/verify_log.txt`) và để runbook phân loại sự cố.
- **Tái lập:** khóa thử nghiệm được lưu một lần trong `keys/`, artifact sinh bằng seed cố định, mọi thời gian mô phỏng là hằng số; `run_all.sh --fresh` sinh lại toàn bộ.

4.4. Ví dụ sử dụng

```
python -m sigeval.cli keygen --alg ed25519 --out keys/release
python -m sigeval.cli sign   --dir artifacts/original --key keys/release --alg ed25519
\
                             --name demo-app --release 1.0.0 --out bundle.json
python -m sigeval.cli trustlist --version 1 --issued 2026-01-01T00:00:00Z \
                             --expires 2026-04-01T00:00:00Z --key
ed25519:keys/release.pub.pem \
                             --root keys/root --out trustlist.json
python -m sigeval.cli verify --dir artifacts/original --bundle bundle.json \
                             --trustlist trustlist.json --root-pub keys/root.pub.pem
# PASS OK                    (sửa 1 byte trong demo-app.bin ->  FAIL HASH_MISMATCH)
```

4.5. Kiểm thử tự động

Bộ pytest gồm 10 kiểm thử: vòng ký–xác minh và âm tính cho cả 4 cấu hình thuật toán (khóa khác, thông điệp khác, chữ ký bị lật bit), PAE ràng buộc alg/key_id, chuẩn hóa không phụ thuộc thứ tự khóa, tính tất định của Ed25519 so với ECDSA, kiểm chứng chéo ECDSA đồ chơi với thư viện cùng khôi phục khóa, và hai kiểm thử tích hợp chạy lại toàn bộ E3, E5, E6 rồi khẳng định mọi ca đúng kỳ vọng. Lần chạy cuối: 10/10 đạt.

CHƯƠNG 5. THỰC NGHIỆM VÀ KẾT QUẢ

5.1. E1 – Bộ artifact

Bảng 9. Bộ artifact dùng cho E3 (sinh bằng seed cố định; nằm ở `artifacts/original`).

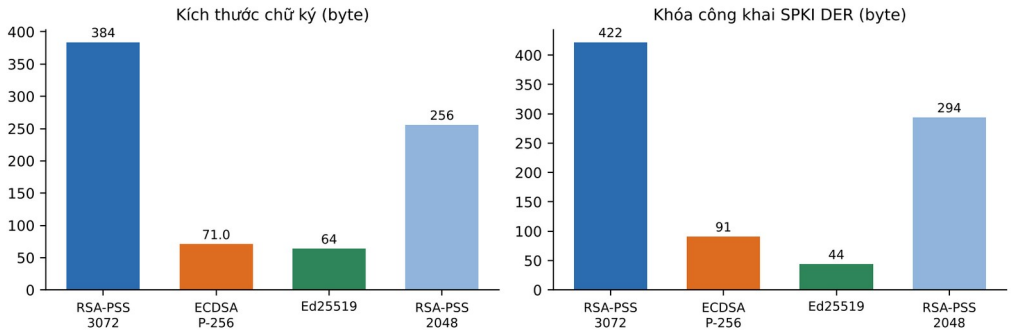
Tệp	Kích thước (byte)	SHA-256 (16 hex đầu)	Loại
`bien-ban.txt`	6,955	5fcb8850ae27f37f…	Cấu hình
`conf/app.json`	59	81f7fb01e7f7eda0…	Tệp nhị phân mô phỏng
`demo-app.bin`	65,544	76b3b0a966a0c161…	Tài liệu văn bản

Tệp nhị phân gồm tiêu đề giả ELF và 64 KiB byte ngẫu nhiên có seed; tài liệu mô phỏng một biên bản nghiệm thu có số tiền (để dễ hình dung hậu quả khi bị sửa). Manifest được dựng từ thư mục này, tệp được sắp theo đường dẫn. Các bản bị sửa mẫu nằm ở `artifacts/tampered-binary` và `artifacts/tampered-document`.

5.2. E2 – Kích thước và hiệu năng

Bảng 10. Kích thước khóa và chữ ký (300 chữ ký ngẫu nhiên mỗi thuật toán).

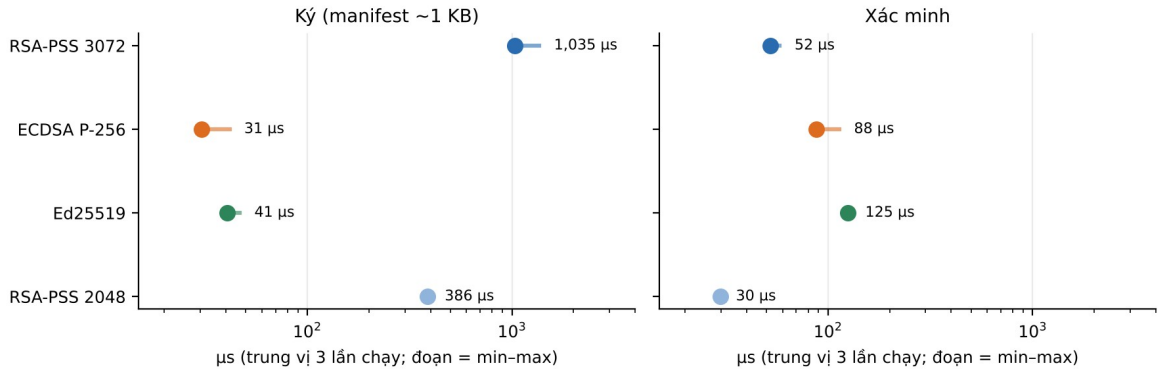
Thuật toán	Khóa riêng PKCS#8 (B)	Khóa công khai SPKI (B)	Chữ ký (B)	Ghi chú
RSA-PSS 3072	1793	422	384.0	= độ dài modulus
ECDSA P-256	138	91	69–72 (TB 70.98)	DER, độ dài đổi theo r, s
Ed25519	48	44	64.0	Cố định
RSA-PSS 2048 (đối chứng)	1218	294	256.0	≈ 112 bit



Hình 2. Kích thước chữ ký và khóa công khai.

Bảng 11. Thời gian trung vị của 3 lần chạy (trong ngoặc: min–max giữa các lần), payload ≈ 1 KB.

Thuật toán	Sinh khóa (µs)	Ký (µs)	Xác minh (µs)	Ops/s: ký / xác minh
RSA-PSS 3072	239,762	1,035.2 (1,014–1,361)	52.4 (52–58)	966 / 19,072
ECDSA P-256	16	30.6 (30–42)	87.5 (87–114)	32,730 / 11,428
Ed25519	35	40.6 (39–47)	124.8 (120–129)	24,602 / 8,016
RSA-PSS 2048	63,156	386.5 (385–398)	29.8 (29–30)	2,588 / 33,542



Hình 3. Thời gian ký và xác minh (thang log). Điểm = trung vị của 3 lần chạy; đoạn = min–max giữa các lần.

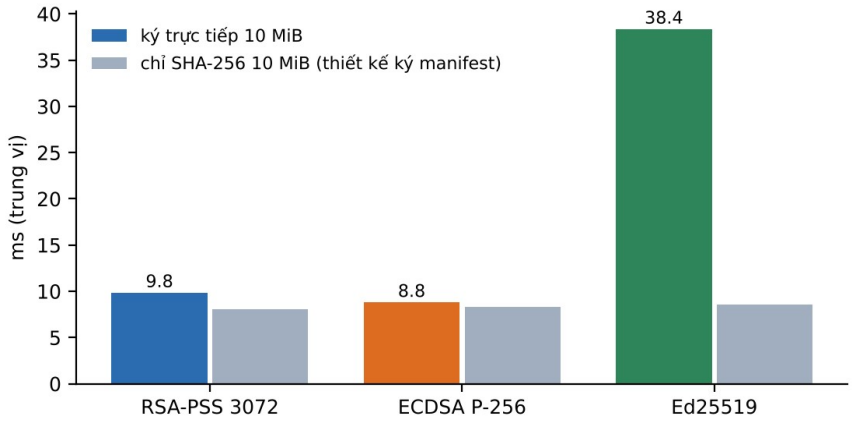
Nhận xét. (i) Về kích thước, Ed25519 gọn nhất (khóa công khai 44 B, chữ ký 64 B); RSA-PSS 3072 có chữ ký lớn gấp 6 lần và khóa công khai gấp gần 10 lần, nhưng với manifest vài KB phần chênh lệch này không đáng kể trong một bó phát hành. (ii) Ký: RSA-PSS 3072 mất 1,035 µs, gấp 34 lần ECDSA (31 µs) và 25 lần Ed25519 (41 µs). (iii) Xác minh: ngược lại, RSA-PSS nhanh nhất (52 µs) nhờ số mũ công khai nhỏ, ECDSA 88 µs, Ed25519 125 µs. Với mô hình "ký một lần, xác minh nhiều lần", RSA-PSS có lợi thế về CPU phía người dùng; đổi lại là băng thông và dung lượng. (iv) **Điều đáng chú ý:** trên môi trường này Ed25519 *không* nhanh hơn ECDSA P-256 (ký 1.33×, xác minh 1.43× thời gian của ECDSA). Nhóm chưa kiểm chứng nguyên nhân; một giả thuyết hợp lý là cài đặt P-256 trong OpenSSL được tối ưu bằng mã hợp ngữ chuyên biệt, trong khi Ed25519 dùng đường mã tổng quát hơn. Kết luận "Ed25519 nhanh hơn" từ tài liệu [9] là kết quả của phần cứng và cài đặt thời điểm đó và không nên coi là hằng đúng.

Độ tin cậy của số đo. Độ lệch giữa các lần chạy (hệ số biến thiên của trung vị) dao động từ 1.5% đến 24.3%; các phép tính cỡ chục µs (ECDSA) nhiễu nhiều nhất. Thời gian đo bao gồm chi phí gọi từ Python (vài µs) nên làm tỉ lệ giữa các phép nhanh bị nén lại. Chỉ nên đọc kết quả theo bậc độ lớn và thứ hạng, không theo con số tuyệt đối; thử nghiệm chạy trên một máy ảo 2 vCPU chia sẻ, một luồng.

5.2.1. Ký tệp lớn: ký trực tiếp so với ký manifest

Bảng 12. Ký trực tiếp tệp lớn (trung vị, 1 lần chạy 5 mẫu).

Thuật toán	Ký trực tiếp 1 MiB (ms)	Ký trực tiếp 10 MiB (ms)	Xác minh trực tiếp 10 MiB (ms)	SHA-256 10 MiB (ms)
RSA-PSS 3072	1.97	9.8	8.8	8.1
ECDSA P-256	0.82	8.8	8.7	8.3
Ed25519	3.57	38.4	19.0	8.6



Hình 4. Ký trực tiếp 10 MiB so với chỉ băm SHA-256 10 MiB.

Với RSA-PSS và ECDSA, thư viện băm SHA-256 trước nên chi phí ký tệp lớn gần bằng chi phí băm (9.8 và 8.8 ms so với 8.3 ms băm). Ed25519 "thuần" xử lý toàn bộ thông điệp trong một lượt và băm hai lần bằng SHA-512 nên mất 38.4 ms, gấp khoảng 4.5 lần chi phí băm SHA-256 (và buộc nạp cả tệp vào bộ nhớ). Đây là lý do thực tế để thiết kế **ký manifest thay vì ký tệp**: chi phí chữ ký trở thành hằng số theo kích thước artifact, phần còn lại là băm từng tệp bằng SHA-256 vốn được tăng tốc phần cứng, và việc băm có thể thực hiện theo luồng.

5.3. E3 – Sửa từng thành phần và hành vi xác minh

Bảng 13. Kết quả E3: 19 ca; mỗi ca chạy với RSA-PSS 3072, ECDSA P-256 và Ed25519 (57 lần xác minh, đều đúng kỳ vọng).

Ca	Thao tác sửa đổi	Thành phần	Kết quả (cả 3 thuật toán)
T00	Đối chứng dương: artifact và bundle nguyên vẹn	(không sửa)	`OK`
T01	Lật 1 bit trong tệp nhị phân	artifact: demo-app.bin	`HASH_MISMATCH`
T02	Thêm 1 ký tự vào tài liệu	artifact: bien-ban.txt	`HASH_MISMATCH`
T03	Sửa metadata builder	manifest.provenance.builder	`BAD_SIGNATURE`
T04	Sửa số phiên bản phát hành	manifest.release	`BAD_SIGNATURE`
T05	Sửa commit nguồn (provenance)	manifest.provenance.source .commit	`BAD_SIGNATURE`
T06	Sửa manifest_version (hạ cấp)	manifest.manifest_version	`BAD_SIGNATURE`
T07	Sửa tệp rồi tính lại băm trong manifest	artifact + manifest.artifacts	`BAD_SIGNATURE`
T08	Lật 1 bit trong chữ ký	signature	`BAD_SIGNATURE`
T09	Đổi key_id sang khóa tin cậy khác cùng thuật toán	bundle.key_id	`BAD_SIGNATURE`
T10	Đổi nhãn thuật toán trong bundle	bundle.alg	`ALG_MISMATCH`
T11	Đổi nhãn thuật toán và key_id không tồn tại	bundle.alg + key_id	`UNKNOWN_KEY`
T12	Xóa một tệp khỏi gói	artifact: bien-ban.txt	`MISSING_FILE`
T13	Chèn thêm tệp không có trong manifest	artifact: backdoor.sh	`UNLISTED_FILE`
T14	Gỡ toàn bộ chữ ký	signatures	`NO_SIGNATURE`
T15	Bundle ký bằng khóa kẻ tấn công (không trong trust list)	signer	`UNKNOWN_KEY`

Ca	Thao tác sửa đổi	Thành phần	Kết quả (cả 3 thuật toán)
T16	Sửa thời gian trong dấu thời gian	signature.timestamp	`TS_INVALID`
T17	Đổi thứ tự khóa/khoảng trắng JSON của manifest (vô hại)	manifest (định dạng)	`OK`
T18	Sửa trường ngoài phạm vi ký (unprotected.notes)	bundle.unprotected	`OK`



Hình 5. Ma trận kết quả E3. Xanh: chấp nhận; đỏ: từ chối với mã lỗi. Không có ô nào khác kỳ vọng.

Phân tích. (1) Có hai lớp phát hiện độc lập: sửa *nội dung tệp* (T01, T02) bị lớp băm bắt ở bước cuối (`HASH_MISMATCH`), trong khi sửa *manifest hoặc metadata* (T03–T07) bị lớp chữ ký bắt trước khi đụng tới tệp (`BAD_SIGNATURE`). T07 là ca quan trọng: kẻ tấn công sửa tệp rồi tính lại băm trong manifest cho khớp; chỉ chữ ký trên manifest mới ngăn được điều này. (2) Nhầm lẫn khóa và thuật toán (T09–T11) bị chặn bởi hai cơ chế khác nhau: PAE ràng buộc alg và key_id vào dữ liệu ký, và trust list ghim kiểu thuật toán của từng khóa. (3) Việc từ chối tệp thừa (T13) là quyết định thiết kế có chủ đích: nếu chỉ kiểm tra các tệp có trong manifest thì kẻ tấn công có thể thả thêm một tệp vào thư mục cài đặt (ví dụ thư viện bị nạp) mà không phá chữ ký nào. (4) T17 cho thấy chuẩn hóa JSON hấp thụ khác biệt định dạng vô hại, không gây từ chối nhầm.

Ca T18 – ranh giới của bảo vệ. Sửa trường `unprotected.notes` *không* làm xác minh thất bại, đúng như thiết kế. Ý nghĩa: chữ ký chỉ bảo vệ những gì nằm trong phạm vi được ký; việc thuật toán mạnh đến đâu không thay đổi điều này. Quy tắc rút ra: mọi dữ liệu dùng để ra quyết định hoặc hiển thị cho người dùng như thông tin đáng tin phải nằm trong manifest; bộ

xác minh phải bỏ qua phần ngoài phạm vi ký. Đây cũng là nguyên nhân của một lớp lỗ hổng thực tế khi giao diện hiển thị trường chưa được ký.

Giới hạn của E3. Kỳ vọng do chính nhóm viết nên một lỗi hiểu nhầm có thể lọt cả ở mã lẫn ở kỳ vọng. Lớp mật mã nguyên thủy được bảo vệ một phần nhờ E7 (đối chiếu độc lập với OpenSSL); lớp chính sách (trust list, dấu thời gian) chỉ được kiểm chứng bằng kiểm thử của nhóm. Chưa thực hiện kiểm thử đột biến (cố ý chèn lỗi vào bộ xác minh) hay fuzzing định dạng bundle.

5.4. E4 – Nonce: tính tất định và tái sử dụng nonce

Bảng 14. Tính tất định của chữ ký.

Cấu hình	Số chữ ký khác nhau trong 20 lần ký cùng thông điệp	Tất định?
RSA-PSS 3072 (muối ngẫu nhiên)	20	Không
ECDSA P-256 (nonce ngẫu nhiên)	20	Không
ECDSA P-256 RFC 6979 (`deterministic_signing=True`)	1	Có (chữ ký vẫn xác minh đúng)
Ed25519	1	Có

Với Ed25519, nửa đầu chữ ký (điểm `R = r·B`) trùng khi ký lại cùng thông điệp và khác khi đổi thông điệp (R_0 = `a489a35cb25333c3`…, R_1 = `aff3de888e6ede39`…), xác nhận nonce là hàm tất định của (khóa, thông điệp).

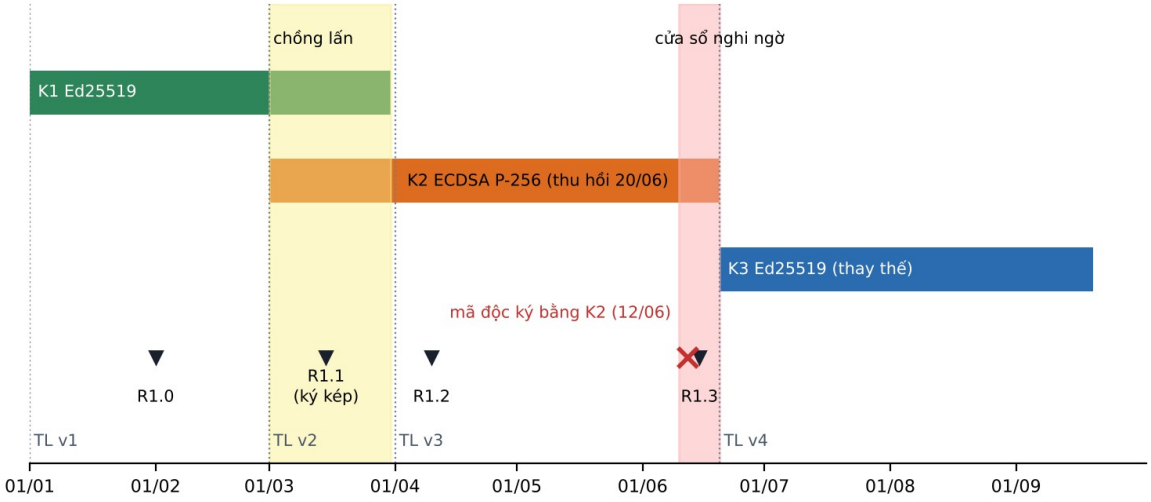
Tái sử dụng nonce ECDSA (chỉ trên khóa thử nghiệm). Nhóm sinh khóa nạn nhân P-256, dùng `toyecdsa.py` ký hai thông điệp với cùng nonce `k`, rồi áp dụng công thức ở mục 2.4:

Bảng 15. Kết quả tấn công tái sử dụng nonce.

Phép kiểm	Kết quả
Hai chữ ký đồ chơi được thư viện chuẩn xác minh bằng khóa công khai nạn nhân	Có (cả hai)
Giá trị r của hai chữ ký trùng nhau	Có
Khóa riêng khôi phục từ (r, s_1, s_2, M_1, M_2) bằng khóa riêng thật	Có
Chữ ký giả mạo bằng khóa khôi phục được xác minh dưới khóa công khai nạn nhân	Có
Đối chứng: hai nonce khác nhau cho cùng r?	Không (công thức không áp dụng được)

Kết quả khẳng định điều bài giảng nêu: chỉ cần hai chữ ký dùng chung nonce là đủ để giả mạo mọi chữ ký tiếp theo, mà không phải phá bài toán logarit rời rạc. Cần nhấn mạnh rằng thư viện chuẩn sinh nonce an toàn; rủi ro nằm ở môi trường: RNG bị sao chép sau khi nhân bản máy ảo, RNG sau khởi động chưa đủ entropy, hay mã tự cài đặt. Phòng thủ gồm dùng Ed25519 hoặc ECDSA tất định (RFC 6979), ký trong HSM/KMS, và giám sát trùng giá trị `r` trên các chữ ký đã phát hành.

5.5. E5 – Luân chuyển khóa với giai đoạn chồng lấn



Hình 6. Dòng thời gian mô phỏng: K1 → K2 (chồng lấn 01–31/03), thu hồi K2 và chuyển sang K3 sau sự cố (E6). TL v1–v4 là các phiên bản trust list.

Kịch bản dùng ba khóa với thuật toán khác nhau để thể hiện tính linh hoạt về thuật toán (K1 Ed25519, K2 ECDSA P-256, K3 Ed25519). Bốn trust list: v1 (chỉ K1), v2 (K1 và K2 cùng hiệu lực), v3 (K1 nghỉ hưu 31/03, K2 hiệu lực), v4 (sau sự cố, xem 5.6). Ba bản phát hành: 1.0.0 (K1, 01/02), 1.1.0 (ký kép K1 và K2, 15/03), 1.2.0 (chỉ K2, 10/04). Mọi chữ ký có dấu thời gian trừ ca R07.

Bảng 16. Kết quả E5 (12 kịch bản, đều đúng kỳ vọng).

Ca	Ngày	Trust list client	Bản	Kết quả	Diễn giải
R01	02/10	v1	1.0.0	OK	Cơ sở: K1 duy nhất
R02	03/20	v1	1.1.0	OK	Client CHƯA cập nhật vẫn xác minh được nhờ chữ ký K1 (chồng lấn)
R03	03/20	v2	1.1.0	OK	Client đã cập nhật: chấp nhận cả hai chữ ký
R04	04/15	v3	1.2.0	OK	Chỉ ký K2 sau khi K1 nghỉ hưu
R05	04/15	v1	1.2.0	UNKNOWN_KEY	Client kẹt ở v1: fail-closed, buộc cập nhật trust list
R06	06/01	v3	1.0.0	OK	K1 đã nghỉ hưu nhưng chữ ký có dấu thời gian 02-01 nằm trong hiệu lực
R07	06/01	v3	1.0.0	KEY_EXPIRED	Cùng bản phát hành nhưng KHÔNG có dấu thời gian: bị từ chối
R08	06/01	v3	1.1.0	OK	Bản ký kép vẫn xác minh qua K2 (và K1 nhờ dấu thời gian)
R09	06/01	v3	1.2.0	OK	

Ca	Ngày	Trust list client	Bản	Kết quả	Diễn giải
R10	06/01	v1	1.2.0	`TRUSTLIST_EXPIRED`	Trust list v1 hết hạn 05-01: client không cập nhật bị từ chối thay vì tin mù
R11	06/01	v1 phát lại	1.2.0	`TRUSTLIST_ROLLBACK`	Kẻ tấn công phát lại v1 cho client đã thấy v3 (nhưng v1 đã hết hạn, kiểm tra rollback chạy trước hạn dùng)
R12	08/15	v3	1.2.0	`TRUSTLIST_EXPIRED`	Trust list v3 hết hạn 07-30

Phân tích. (a) Chồng lấn có tác dụng: client chưa cập nhật (R02) vẫn xác minh bản ký kép nhờ chữ ký của khóa cũ, không phải buộc mọi người cập nhật cùng lúc. (b) Hết chồng lấn, client lạc hậu bị từ chối (R05, R10) – hỏng theo hướng an toàn nhưng có chi phí hỗ trợ người dùng, nên thời gian chồng lấn phải đủ dài so với chu kỳ cập nhật của người dùng. (c) Khóa nghỉ hưu vẫn xác minh được bản cũ **nếu và chỉ nếu** có dấu thời gian (R06 so với R07): cùng một bản phát hành, chỉ khác dấu thời gian, cho hai kết quả đối lập. (d) Trust list đóng vai trò "hạn dùng của niềm tin": R10 và R12 cho thấy client không cập nhật bị từ chối khi trust list hết hạn thay vì tin mù; R11 cho thấy phát lại phiên bản cũ bị chặn nhờ số phiên bản đơn điệu.

5.6. E6 – Khóa ký bị lộ

Kịch bản: khóa K2 bị lộ; thời điểm nghi ngờ sớm nhất là 10/06 (`compromised_since`); kẻ tấn công ký mã độc bản "1.2.1" ngày 12/06; đội phát hành phát hiện và phát hành trust list v4 ngày 20/06 (thu hồi K2, kích hoạt K3). Đội phát hành cũng đã ký bản hợp lệ 1.3.0 bằng K2 ngày 15/06, nằm trong cửa sổ nghi ngờ.

Bảng 17. Kết quả E6 (9 kịch bản, đều đúng kỳ vọng).

Ca	Ngày	Trust list	Đối tượng	Kết quả	Diễn giải
C01	06/21	v4	1.2.0	`OK`	Bản 1.2.0 ký K2 với dấu thời gian 04-10 < compromised_since 06-10: giữ hiệu lực
C02	06/21	v4	1.3.0	`REVOKED`	Bản hợp lệ 1.3.0 ký K2 ngày 06-15 nằm trong cửa sổ nghi ngờ: phải ký lại
C03	06/21	v4	1.3.0	`OK`	1.3.0 ký lại bằng K3 (trust list v4)
C04	06/21	v4	1.2.1 (mã độc)	`REVOKED`	Mã độc ký K2, có dấu thời gian THẬT từ TSA (06-12 >= 06-10)
C05	06/21	v4	1.2.1 (mã độc)	`REVOKED`	Mã độc ký K2, không dấu thời gian

Ca	Ngày	Trust list	Đối tượng	Kết quả	Diễn giải
C06	06/21	v4	1.2.1 (mã độc)	`TS_INVALID`	Kẻ tấn công tự ký dấu thời gian lùi về 04-01: TSA không tin cậy
C07	06/15	v3	1.2.1 (mã độc)	`OK`	RỦI RO DƯ: client còn trust list v3 (trước thu hồi) CHẤP NHẬN mã độc
C08	06/21	v3	1.2.1 (mã độc)	`OK`	Client chưa nhận v4 sau 1 ngày: vẫn nằm trong cửa sổ phơi nhiễm đến khi cập nhật/hết hạn v3 (07-30)
C09	06/21	v3 phát lại	1.2.1 (mã độc)	`TRUSTLIST_ROLLBACK`	Phát lại trust list v3 cho client đã thấy v4

Phân tích. (1) Chính sách "chấp nhận chữ ký có dấu thời gian trước `compromised_since`" giữ được hiệu lực cho bản phát hành cũ (C01) mà không cần ký lại hàng loạt, đồng thời loại mã độc dù kẻ tấn công xin được dấu thời gian **thật** tại thời điểm hiện tại (C04) hoặc không có dấu (C05). (2) Kẻ tấn công không thể lùi thời gian: dấu thời gian tự ký bị từ chối vì TSA không tin cậy (C06). Điều này đúng *với giả định TSA không bị chiếm* (A5). (3) Cái giá: bản hợp lệ ký trong cửa sổ nghi ngờ không phân biệt được với mã độc (C02) và phải ký lại bằng khóa mới (C03); do đó `compromised_since` nên đặt bảo thủ (sớm), đánh đổi số bản phải ký lại. (4) **Rủi ro dư quan trọng nhất:** client chưa nhận v4 vẫn chấp nhận mã độc (C07, C08). Thu hồi chỉ hiệu quả khi trust list mới đến được client. Phơi nhiễm tối đa của một client lạc hậu bằng khoảng thời gian từ lúc khóa bị lộ đến khi trust list cũ hết hạn: trong mô phỏng, 10/06 → 30/07 là 50 ngày (40 ngày kể từ khi v4 được phát hành). (5) Chặn phát lại trust list cũ chỉ hiệu quả với client đã từng thấy v4 (C09); client chưa từng thấy phiên bản mới hơn không có cách tự phát hiện.

Bảng 18. Đánh đổi giữa hạn dùng trust list và cửa sổ phơi nhiễm (suy luận từ mô hình, không phải số đo).

TTL trust list	Phơi nhiễm tối đa sau khi v4 phát hành (client lạc hậu)	Chi phí vận hành
≈120 ngày (như mô phỏng)	Đến 120 ngày	Thấp; client ít phải liên lạc
30 ngày	Đến 30 ngày	Trung bình
7 ngày	Đến 7 ngày	Cao; cần kênh cập nhật tin cậy, offline lâu sẽ bị từ chối

5.7. E7 – Kiểm chứng chéo với OpenSSL

Bảng 19. Kết quả liên thông (OpenSSL 3.0.13 CLI so với backend OpenSSL của thư viện Python).

Thuật toán	Python ký → OpenSSL xác minh	OpenSSL ký → Python xác minh	OpenSSL từ chối bản bị sửa
RSA-PSS 3072	Đạt	Đạt	Đạt
ECDSA P-256	Đạt	Đạt	Đạt

Thuật toán	Python ký → OpenSSL xác minh	OpenSSL ký → Python xác minh	OpenSSL từ chối bản bị sửa
Ed25519	Đạt	Đạt	Đạt

Cả ba thuật toán liên thông hai chiều với cùng tham số (RSA-PSS: SHA-256, MGF1-SHA-256, muối 32 B; ECDSA: SHA-256, DER; Ed25519: `-rawin`). Việc này kiểm chứng rằng bộ tham số nhóm công bố đủ để cài đặt độc lập, và là kiểm chứng độc lập duy nhất cho lớp nguyên thủy.

CHƯƠNG 6. THẢO LUẬN, GIỚI HẠN VÀ KHUYẾN NGHỊ

6.1. Trả lời các câu hỏi nghiên cứu

Bảng 20. Tổng hợp kết quả theo câu hỏi nghiên cứu.

Câu hỏi	Kết luận từ thực nghiệm	Mức chắc chắn
RQ1	Ed25519 gọn nhất (chữ ký 64 B); RSA-PSS 3072 ký chậm hơn ECDSA ≈34 lần nhưng xác minh nhanh nhất; Ed25519 không nhanh hơn ECDSA trên môi trường này. ECDSA đòi hỏi nonce duy nhất và không thiên lệch (đã chứng minh khôi phục khóa khi trùng); Ed25519 và ECDSA RFC 6979 tất định; PSS dùng muối ngẫu nhiên không nguy hiểm khi muối lặp.	Cao về kích thước và nonce; trung bình về hiệu năng (một máy, nhiễu 1–24%)
RQ2	Mọi sửa đổi trong phạm vi bảo vệ bị từ chối với mã đúng nguyên nhân; trường ngoài phạm vi ký không được bảo vệ.	Cao cho các ca đã thử; chưa phủ fuzzing
RQ3	Chồng lấn + trust list có phiên bản cho phép đổi khóa mượt; client lạc hậu hỏng theo hướng an toàn.	Cao trong mô hình mô phỏng
RQ4	Dấu thời gian + compromised_since giữ hiệu lực bản cũ và loại mã độc; client chưa cập nhật vẫn bị phơi nhiễm đến khi trust list hết hạn.	Cao về logic; phụ thuộc giả định TSA

6.2. Những điều thực nghiệm cho thấy về thiết kế

- **Thuật toán không phải điểm yếu chính.** Cả ba thuật toán cho kết quả giống hệt trong E3, E5, E6; sự khác biệt hành vi nằm ở nonce (E4) và hiệu năng (E2). Các lỗ hổng thực sự của hệ thống nằm ở phạm vi chữ ký (T18), độ "tươi" của trust list (C07/C08) và độ tin cậy của TSA.
- **Thu hồi luôn có độ trễ.** Không cơ chế nào trong mô hình offline xóa được cửa sổ giữa lúc khóa bị lộ và lúc mọi client biết. Có thể rút ngắn bằng TTL ngắn của trust list, nhưng đánh đổi độ sẵn sàng (bảng ở 5.6). Điều này phù hợp với hạn chế đã biết của CRL (chậm, nặng) so với OCSP stapling (nhanh, cần máy chủ) trong bài giảng.
- **Dấu thời gian là điều kiện để khóa cũ có ý nghĩa.** Nếu không có, nghỉ hưu hay thu hồi khóa đều buộc ký lại toàn bộ lịch sử hoặc chấp nhận vô hiệu hóa chúng (R07, C05).
- **Ký kép là chi phí nhỏ cho độ bền lớn.** Ký kép K1+K2 chỉ thêm một chữ ký (64–384 B trước khi mã hóa base64) mà giữ được toàn bộ client cũ trong giai đoạn chồng lấn.

6.3. Khuyến nghị chọn thuật toán

Bảng 21. Khuyến nghị theo bối cảnh (không phải đánh giá tuyệt đối).

Bối cảnh	Khuyến nghị	Lý do (dựa trên số đo của báo cáo)
Phát hành phần mềm cho nhiều người dùng, môi trường cũ, chấp nhận gói lớn hơn	RSA-PSS 3072	Xác minh nhanh nhất (≈52 µs); hỗ trợ rộng; muối lặp không gây thảm họa. Chữ ký 384 B.
Hệ thống mới, thiết bị nhúng, RNG chưa chắc tin cậy, cần chữ ký gọn	Ed25519	Nonce tất định loại bỏ rủi ro RNG; chữ ký 64 B. Lưu ý: ký trực tiếp tệp lớn chậm; luôn ký manifest.
Phải tương thích HSM/PKI/tiêu chuẩn đã có ECDSA	ECDSA P-256	Ký nhanh nhất (≈31 µs). Bắt buộc quản lý nonce (HSM hoặc RFC 6979) và giám sát trùng r.

Dù chọn thuật toán nào, khuyến nghị chung là: ký manifest thay vì tệp; ràng buộc alg và key_id vào dữ liệu ký; ghim thuật toán ở trust list; luôn kèm dấu thời gian; đặt TTL trust list ngắn nhất mà kênh cập nhật chịu được; ký kép khi luân chuyển.

6.4. Giới hạn của nghiên cứu

- **Hiệu năng:** một máy ảo 2 vCPU chia sẻ, 3 lần chạy; chưa đo trên nhiều CPU, chưa đo bộ nhớ, chưa đo hằng thời gian (side channel). Chưa lý giải nguyên nhân Ed25519 chậm hơn kỳ vọng.
- **TSA mô phỏng** cho phép đặt thời gian tùy ý, không phải RFC 3161, không có chuỗi chứng thư; giả định TSA trung thực (A5) chưa được thử thách.
- **Không có CA/X.509 thật, không có OCSP/CRL thật:** trust list là mô hình thu nhỏ; chưa kiểm tra phân tích đường dẫn chứng thư, ràng buộc tên, EKU.
- **Khóa riêng lưu ở dạng PEM không mã hóa** vì là khóa thử nghiệm; hệ thống thật cần HSM/KMS, quy trình offline cho khóa gốc.
- **Chưa mô hình hóa tấn công trước khi ký** (A6: hệ thống build bị chiếm). Chữ ký trên mã độc do pipeline sinh ra vẫn hợp lệ; provenance trong manifest chỉ là khai báo.
- **Kỳ vọng kiểm thử do nhóm tự viết**, chưa có kiểm thử đột biến hay fuzzing; chưa có phân tích chống lại tấn công thời gian (timing) trong so sánh băm.
- **Phạm vi pháp lý:** nguyên mẫu không tạo chữ ký số có giá trị pháp lý theo quy định hiện hành của Việt Nam.

6.5. Hướng cải tiến

- Dùng HSM/KMS (hoặc khóa phần cứng) cho khóa ký và khóa gốc; ký ngưỡng (nhiều người phê duyệt) cho trust list.
- Thêm nhật ký minh bạch (transparency log) kiểu Sigstore/Rekor để phát hiện ký trái phép ngay cả khi khóa bị lộ [23]; gắn tuyên bố provenance theo in-toto/SLSA [15], [22].
- Hiện thực TSA theo RFC 3161 với chuỗi chứng thư, và OCSP stapling cho phần chứng chỉ.

- **Linh hoạt thuật toán và hậu lượng tử:** RSA và ECDSA/EdDSA đều bị phá bởi máy tính lượng tử đủ lớn. NIST đã chuẩn hóa ML-DSA (FIPS 204) [26]; thiết kế trust list có nhãn thuật toán sẵn sàng cho chữ ký kép (cổ điển + hậu lượng tử) trong giai đoạn chuyển tiếp.
- Kiểm thử đột biến và fuzzing bộ phân tích bundle/manifest; mở rộng E2 sang nhiều máy và đo bộ nhớ.

CHƯƠNG 7. RUNBOOK XỬ LÝ KHÓA KÝ BỊ LỘ

Runbook này là sản phẩm bắt buộc của đề tài; các bước được kiểm chứng ở E6 (mã ca nêu trong ngoặc). Bản đầy đủ ở tệp `RUNBOOK.md`. Vai trò: **IC** (điều phối sự cố), **KC** (người giữ khóa), **RE** (kỹ sư phát hành), **CM** (truyền thông).

Bảng 22. Runbook xử lý khóa ký bị lộ.

Bước	Hành động	Vai trò	Mục tiêu thời gian	Bằng chứng / kiểm chứng
0	Phát hiện: chữ ký lạ trên log, báo cáo của người dùng, nghi ngờ lộ thiết bị chứa khóa. Mở sự cố, ghi giờ phát hiện.	IC	T+0	Phiếu sự cố
1	Dừng ký: khóa K_cũ bị vô hiệu hóa trong CI/HSM; đóng băng phát hành.	KC, RE	≤ 30 phút	Nhật ký HSM/CI
2	Xác định `compromised_since`: thời điểm sớm nhất có thể khóa bị lộ (bảo thủ). Liệt kê mọi bản đã ký sau mốc này và các bản không có dấu thời gian.	IC, RE	≤ 4 giờ	Danh sách bản phát hành
3	Sinh khóa mới K_mới offline bằng CSPRNG/HSM; ghi dấu vân tay; hai người xác nhận qua kênh ngoài băng.	KC	≤ 4 giờ	Dấu vân tay công khai
4	Phát hành trust list `v+1` ký bằng khóa gốc: K_cũ = revoked + compromised_since; K_mới = active; **TTL ngắn**. Số phiên bản tăng đơn điệu. (C02, C04, C09)	KC	≤ 8 giờ	Tệp trust list, nhật ký ký
5	Ký lại bằng K_mới các bản có dấu thời gian ≥ mốc nghi ngờ hoặc không có dấu thời gian; bản có dấu thời gian trước mốc được giữ nhưng rà soát nội dung. (C01, C02, C03)	RE	≤ 24 giờ	Bundle mới; log xác minh
6	Xác minh thử bằng client sạch ở đúng trạng thái `v+1`: bản cũ trước mốc đạt; bản mã độc/giả dấu thời gian bị từ chối. (C04–C06)	RE	Trước khi công bố	Log xác minh
7	Công bố: đẩy trust list qua mọi kênh (CDN, ứng dụng, thông báo bảo mật) và cảnh báo người dùng; liệt kê dấu vân tay hợp lệ mới.	CM, IC	≤ 24 giờ	Bản tin; mẫu cảnh báo bên dưới

Bước	Hành động	Vai trò	Mục tiêu thời gian	Bằng chứng / kiểm chứng
8	Giám sát client lạc hậu: đo tỉ lệ client còn ở $v \le v_cũ$; kéo dài hỗ trợ, nhắc cập nhật. Nhớ rằng client chưa nhận $v+1$ vẫn chấp nhận mã độc đến khi trust list cũ hết hạn. (C07, C08)	IC, CM	Đến khi trust list cũ hết hạn	Báo cáo tỉ lệ cập nhật
9	Hậu kiểm: tìm nguyên nhân lộ; hủy an toàn khóa riêng cũ; giữ khóa công khai để xác minh bản cũ; cập nhật chính sách (TTL, HSM, giám sát trùng r).	IC, KC	≤ 14 ngày	Báo cáo hậu kiểm

Mẫu cảnh báo người dùng. "Ngày [ngày], chúng tôi phát hiện khóa ký phát hành [dấu vân tay cũ rút gọn] có thể đã bị lộ từ [compromised_since]. Vui lòng cập nhật danh sách tin cậy lên phiên bản [v+1] và chỉ cài đặt các bản có chữ ký từ khóa [dấu vân tay mới rút gọn]. Không cài các bản phát hành mang số phiên bản [danh sách] nếu tải trước khi cập nhật. Dấu vân tay đầy đủ có tại [kênh xác thực]. Nếu không cập nhật được, hãy liên hệ [đầu mối]."

Điều kiện dừng và leo thang. Nếu nghi ngờ **khóa gốc** hoặc **khóa TSA** bị lộ (A5), runbook này không đủ: client phải được cấp lại điểm tin cậy ban đầu (ghim khóa gốc mới) qua kênh ngoài băng, tức là coi như phát hành lại toàn bộ.

KẾT LUẬN

Báo cáo đã xây dựng và đánh giá một nguyên mẫu ký/xác minh artifact với ba lược đồ chữ ký chuẩn, bằng bảy thực nghiệm có số liệu tái lập. Về kỹ thuật: RSA-PSS 3072, ECDSA P-256 và Ed25519 đều phát hiện đầy đủ 57/57 sửa đổi trong phạm vi bảo vệ; khác biệt thực chất nằm ở kích thước (64 B so với 384 B), hiệu năng (RSA-PSS ký chậm ≈34 lần ECDSA nhưng xác minh nhanh nhất; Ed25519 không nhanh hơn ECDSA ở môi trường này) và yêu cầu nonce (ECDSA nhạy cảm tới mức hai chữ ký cùng nonce làm lộ khóa riêng, đã được chứng minh bằng thực nghiệm).

Về quy trình, đóng góp chính là các kết quả định lượng và kịch bản cho phần vòng đời khóa: chồng lấn và trust list có phiên bản cho phép luân chuyển an toàn; dấu thời gian cho phép giữ hiệu lực bản phát hành cũ sau khi khóa bị thu hồi; nhưng thu hồi không bao giờ tức thời – client lạc hậu vẫn chấp nhận mã độc cho đến khi trust list cũ hết hạn. Thông điệp chung khớp với bài giảng: **an toàn của chữ ký số được quyết định bởi quản lý khóa và phạm vi dữ liệu được ký không kém, và nhiều khi hơn, so với việc chọn thuật toán.**

Các giới hạn (TSA mô phỏng, không có PKI thật, một máy đo, kỳ vọng tự viết) đã nêu ở 6.4; kết luận chỉ áp dụng cho mô hình đe dọa ở 3.1. Hướng tiếp theo là HSM, nhật ký minh bạch, TSA theo RFC 3161 và chuẩn bị chữ ký hậu lượng tử.

TÀI LIỆU THAM KHẢO

[1] W. Diffie and M. E. Hellman, “New directions in cryptography,” IEEE Trans. Inf. Theory, vol. 22, no. 6, pp. 644–654, Nov. 1976.

[2] R. L. Rivest, A. Shamir, and L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” Commun. ACM, vol. 21, no. 2, pp. 120–126, 1978.

[3] S. Goldwasser, S. Micali, and R. L. Rivest, “A digital signature scheme secure against adaptive chosen-message attacks,” SIAM J. Comput., vol. 17, no. 2, pp. 281–308, 1988.

[4] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, “PKCS #1: RSA Cryptography Specifications Version 2.2,” IETF RFC 8017, Nov. 2016.

[5] M. Bellare and P. Rogaway, “The exact security of digital signatures – How to sign with RSA and Rabin,” in Advances in Cryptology – EUROCRYPT ’96, LNCS 1070, 1996, pp. 399–416.

[6] National Institute of Standards and Technology, “Digital Signature Standard (DSS),” FIPS PUB 186-5, Feb. 2023.

[7] D. Johnson, A. Menezes, and S. Vanstone, “The Elliptic Curve Digital Signature Algorithm (ECDSA),” Int. J. Inf. Secur., vol. 1, no. 1, pp. 36–63, 2001.

[8] T. Pornin, “Deterministic usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA),” IETF RFC 6979, Aug. 2013.

[9] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, “High-speed high-security signatures,” J. Cryptographic Engineering, vol. 2, no. 2, pp. 77–89, 2012.

[10] fail0verflow (M. Bushing, H. Marcan, et al.), “Console hacking 2010: PS3 epic fail,” 27th Chaos Communication Congress (27C3), Dec. 2010.

[11] N. Heninger, Z. Durumeric, E. Wustrow, and J. A. Halderman, “Mining your Ps and Qs: Detection of widespread weak keys in network devices,” in Proc. 21st USENIX Security Symp., 2012.

[12] S. Josefsson and I. Liusvaara, “Edwards-Curve Digital Signature Algorithm (EdDSA),” IETF RFC 8032, Jan. 2017.

[13] E. Barker, “Recommendation for Key Management: Part 1 – General,” NIST SP 800-57 Part 1, Rev. 5, May 2020.

[14] J. Samuel, N. Mathewson, J. Cappos, and R. Dingledine, “Survivable key compromise in software update systems,” in Proc. ACM CCS, 2010, pp. 61–72.

[15] S. Torres-Arias, H. Weiss, I. Kuppusamy, and J. Cappos, “in-toto: Providing farm-to-table guarantees for bits and bytes,” in Proc. 28th USENIX Security Symp., 2019, pp. 1393–1410.

[16] D. Cooper et al., “Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile,” IETF RFC 5280, May 2008.

[17] S. Santesson et al., “X.509 Internet Public Key Infrastructure Online Certificate Status Protocol – OCSP,” IETF RFC 6960, Jun. 2013.

[18] Quốc hội nước CHXHCN Việt Nam, “Luật Giao dịch điện tử,” Luật số 20/2023/QH15, ngày 22/6/2023.

[19] Chính phủ, Nghị định 130/2018/NĐ-CP hướng dẫn Luật Giao dịch điện tử về chữ ký số và dịch vụ chứng thực chữ ký số; Nghị định 23/2025/NĐ-CP về chữ ký điện tử và dịch vụ tin cậy (nhóm cần đối chiếu văn bản còn hiệu lực).

[20] Python Cryptographic Authority, "cryptography documentation," v50.0.1 (phiên bản dùng trong thực nghiệm).

[21] C. Adams, P. Cain, D. Pinkas, and R. Zuccherato, "Internet X.509 PKI Time-Stamp Protocol (TSP)," IETF RFC 3161, Aug. 2001.

[22] OpenSSF, "Supply-chain Levels for Software Artifacts (SLSA) specification, v1.0," 2023. [Online]. Available: https://slsa.dev

[23] Z. Newman, J. S. Meyers, and S. Torres-Arias, "Sigstore: Software signing for everybody," in Proc. ACM CCS, 2022.

[24] OpenSSL Project, "OpenSSL 3.0 manual pages: openssl-dgst(1), openssl-pkeyutl(1)," https://www.openssl.org/docs/.

[25] Bài giảng An toàn thông tin, Chương 4 – Các cơ chế mật mã hiện đại, Trường ĐH Giao thông vận tải TP.HCM, 2026.

[26] National Institute of Standards and Technology, "Module-Lattice-Based Digital Signature Standard," FIPS PUB 204, Aug. 2024.

PHỤ LỤC A. BẢNG TRUY VẾT MINH CHỨNG

Bảng 23. Bảng truy vết: mỗi mục có thể tái tạo bằng `run_all.sh`.

Mã	Phép thử	Kết quả mong đợi	Kết quả thực tế	Tệp minh chứng	Phụ trách
E1	Tạo bộ artifact + manifest	3 tệp, băm khớp manifest	Đạt	`artifacts/original/`, `artifacts/bundles/*.json`	[điền]
E2a	Kích thước khóa/chữ ký	Ed25519 64 B; RSA-3072 384 B	Đạt	`results/sizes.csv`	[điền]
E2b	Thời gian keygen/ký/xác minh (3 lần)	Có trung vị, min–max	Đã đo	`results/benchmark_run1..3.csv`, `benchmark_summary.csv`, `benchmark_env.json`	[điền]
E2c	Ký tệp lớn 1/10 MiB	Ký trực tiếp vs băm	Đã đo	`results/benchmark_large.csv`	[điền]
E3	19 ca sửa đổi × 3 thuật toán	57/57 đúng kỳ vọng	57/57	`results/tamper_results.csv`, `results/verify_log.txt`, `artifacts/tampered-*`	[điền]
E4	Nonce, tái sử dụng nonce ECDSA	Khôi phục được khóa; Ed25519 tất định	Đạt	`results/nonce_results.json`	[điền]
E5	Luân chuyển khóa (12 ca)	12/12	12/12	`results/rotation_results.csv`, `artifacts/lifecycle/`	[điền]
E6	Khóa bị lộ (9 ca)	9/9	9/9	`results/compromise_results.csv`	[điền]
E7	Liên thông OpenSSL	3/3 thuật toán, 2 chiều	Đạt	`results/interop_openssl.json`	[điền]
K1	Khóa thử nghiệm + dấu vân tay công khai	Có, ghi rõ LAB ONLY	Có	`keys/`, `keys/fingerprints.json`	[điền]
K2	Sơ đồ vòng đời khóa ký	Có	Hình 3 ở Phụ lục E, `figures/f2_key_lifecycle.png`	`figures/`	[điền]
K3	Runbook khóa bị lộ	Có, kèm bằng chứng kiểm thử	Chương 7, `RUNBOOK.md`	`RUNBOOK.md`	[điền]
T	Kiểm thử tự động	10/10 đạt	10/10	`tests/test_sigeval.py`	[điền]

PHỤ LỤC B. ĐÓNG GÓP CỦA THÀNH VIÊN

Theo yêu cầu của bộ đề tài, cả ba thành viên tham gia thiết kế, thực nghiệm, phân tích và bảo vệ. **Nhóm điền thông tin thật vào bảng này trước khi nộp**; các ô dưới đây là khung gợi ý, không phải ghi nhận đã xảy ra.

Bảng 24. Phân công và đóng góp (mẫu cần điền).

Thành viên	Phần phụ trách chính (điền)	Phần cùng tham gia	Tỉ lệ đóng góp
[Họ tên 1]	[vd: mô hình đe dọa, E3, vòng đời khóa]	Thiết kế chung, kiểm thử chéo	[ ]%
[Họ tên 2]	[vd: core/manifest, E2, E4]	Thiết kế chung, kiểm thử chéo	[ ]%
[Họ tên 3]	[vd: trust list/verify, E5, E6, runbook]	Thiết kế chung, kiểm thử chéo	[ ]%

PHỤ LỤC C. CÔNG KHAI SỬ DỤNG AI

Bảng 25. Bảng công khai việc sử dụng AI (nhóm xác nhận/chỉnh sửa trước khi nộp).

Công cụ	Mục đích sử dụng	Nội dung đã kiểm tra	Phần nhóm tự chịu trách nhiệm
Claude (Anthropic)	Hỗ trợ viết mã nguyên mẫu `sigeval`, kịch bản thực nghiệm, biểu đồ và soạn thảo báo cáo này từ số liệu đo.	Chạy lại toàn bộ thực nghiệm; pytest 10/10; đối chiếu bảng số liệu trong báo cáo với tệp CSV/JSON (báo cáo đọc trực tiếp từ các tệp này); kiểm chứng chéo chữ ký với OpenSSL; xem lại biểu đồ.	Hiểu và giải thích được mọi thành phần khi bảo vệ; quyết định phạm vi, tiêu chí chấp nhận và kết luận; điền phân công thực tế; đối chiếu văn bản pháp luật và tài liệu tham khảo với nguồn gốc trước khi nộp.

PHỤ LỤC D. HƯỚNG DẪN TÁI LẬP

```
pip install -r requirements.txt --break-system-packages   # cryptography>=46,
matplotlib, pytest, python-docx
./run_all.sh            # chạy E3–E7, benchmark 3 lần, vẽ hình, pytest   (~1 phút)
./run_all.sh --fresh    # xóa và sinh lại khóa thử nghiệm, artifact, kết quả
python3 report/build_report.py   # dựng lại báo cáo Word từ results/
```

Thời gian mô phỏng và seed là hằng số nên E3–E7 cho kết quả giống nhau trên mọi máy; chỉ số đo hiệu năng (E2) thay đổi theo phần cứng và nên được ghi lại cùng `benchmark_env.json`.

PHỤ LỤC E. DẤU VÂN TAY CÔNG KHAI VÀ TRÍCH LOG

Bảng 26. Dấu vân tay khóa thử nghiệm (đầy đủ 64 hex trong `keys/fingerprints.json`). CHỈ DÙNG TRONG ĐỒ ÁN.

Khóa	Thuật toán	Dấu vân tay (8 byte đầu của SHA-256 SPKI)
Khóa gốc ký trust list	ed25519	`19:02:42:69:85:31:E0:C1`
Khóa TSA mô phỏng	ed25519	`B4:BF:EA:33:F5:F1:E5:3E`
Luân chuyển: K1	ed25519	`96:BB:81:20:D0:76:93:9A`
Luân chuyển: K2 (bị lộ)	ecdsa-p256	`2D:EB:6A:A8:80:A1:C9:F9`
Luân chuyển: K3 (thay thế)	ed25519	`63:A4:0D:98:14:44:0C:B8`
E3: khóa ký RSA-PSS 3072	rsa-pss-3072	`13:1F:3F:F2:94:A4:B8:83`

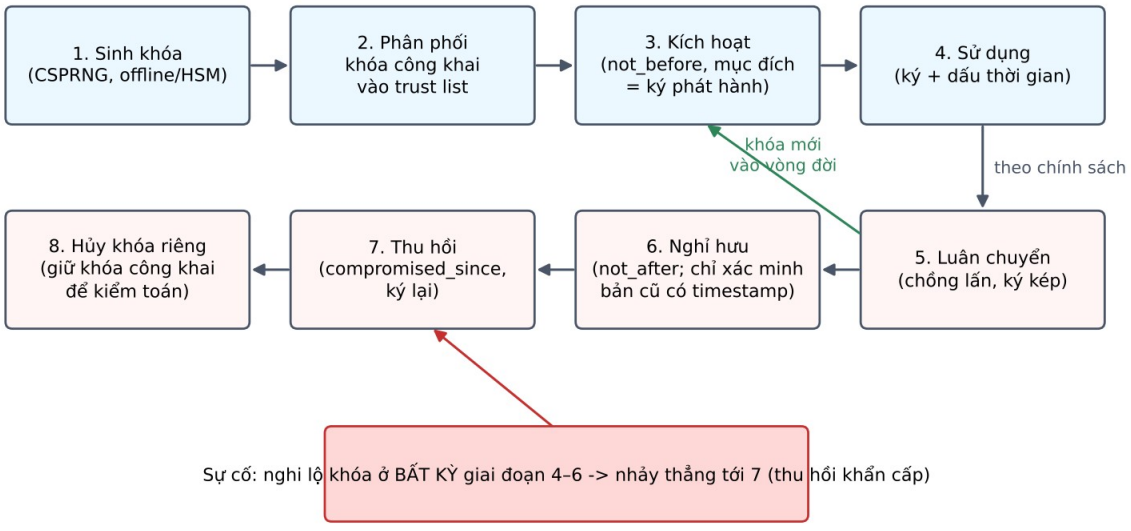
Khóa	Thuật toán	Dấu vân tay (8 byte đầu của SHA-256 SPKI)
E3: khóa ký ECDSA P-256	ecdsa-p256	`05:32:B0:10:D4:21:E4:AE`
E3: khóa ký Ed25519	ed25519	`3C:62:BD:0B:AE:D9:04:BC`

Trích `results/verify_log.txt` (Ed25519):

```
[ed25519] T00 Đối chứng dương: artifact và bundle nguyên vẹn -> ACCEPT code=OK expected=OK OK
[ed25519] T01 Lật 1 bit trong tệp nhị phân -> REJECT code=HASH_MISMATCH expected=HASH_MISMATCH OK demo-
app.bin
[ed25519] T02 Thêm 1 ký tự vào tài liệu -> REJECT code=HASH_MISMATCH expected=HASH_MISMATCH OK bien-
ban.txt
[ed25519] T03 Sửa metadata builder -> REJECT code=BAD_SIGNATURE expected=BAD_SIGNATURE OK
ed25519:3c62bd0b chữ ký toán…
[ed25519] T04 Sửa số phiên bản phát hành -> REJECT code=BAD_SIGNATURE expected=BAD_SIGNATURE OK
ed25519:3c62bd0b chữ k…
[ed25519] T05 Sửa commit nguồn (provenance) -> REJECT code=BAD_SIGNATURE expected=BAD_SIGNATURE OK
ed25519:3c62bd0b ch…
[ed25519] T06 Sửa manifest_version (hạ cấp) -> REJECT code=BAD_SIGNATURE expected=BAD_SIGNATURE OK
ed25519:3c62bd0b ch…
[ed25519] T07 Sửa tệp rồi tính lại băm trong manifest -> REJECT code=BAD_SIGNATURE expected=BAD_SIGNATURE
OK ed25519:3…
[ed25519] T08 Lật 1 bit trong chữ ký -> REJECT code=BAD_SIGNATURE expected=BAD_SIGNATURE OK
ed25519:3c62bd0b chữ ký to…
[ed25519] T09 Đổi key_id sang khóa tin cậy khác cùng thuật toán -> REJECT code=BAD_SIGNATURE
expected=BAD_SIGNATURE OK…
[ed25519] T10 Đổi nhãn thuật toán trong bundle -> REJECT code=ALG_MISMATCH expected=ALG_MISMATCH OK
ecdsa-p256:3c62bd0…
[ed25519] T11 Đổi nhãn thuật toán và key_id không tồn tại -> REJECT code=UNKNOWN_KEY expected=UNKNOWN_KEY
OK ecdsa-p25…
[ed25519] T12 Xóa một tệp khỏi gói -> REJECT code=MISSING_FILE expected=MISSING_FILE OK bien-ban.txt
[ed25519] T13 Chèn thêm tệp không có trong manifest -> REJECT code=UNLISTED_FILE expected=UNLISTED_FILE
OK backdoor.sh
```

Mã nguồn đầy đủ, dữ liệu CSV và log nằm trong kho kèm theo (không tính vào số trang báo cáo).

PHỤ LỤC F. SƠ ĐỒ VÒNG ĐỜI KHÓA KÝ



Hình 7. Vòng đời khóa ký (8 giai đoạn theo bài giảng) và đường tắt sang thu hồi khẩn cấp khi nghi ngờ lộ khóa.